递归方程解的渐近阶的求法

递归算法在最坏情况下的时间复杂性渐近阶的分析，都转化为求相应的一个递归方程的解的渐近阶。因此，求递归方程的解的渐近阶是对递归算法进行分析的关键步骤。

递归方程的形式多种多样，求其解的渐近阶的方法也多种多样。这里只介绍比较实用的五种方法。

重点讲前 3 个方法，后面两个不讲或略讲一次的课。

1. 代入法 这个方法的基本步骤是先推测递归方程的显式解，然后用数学归纳法证明这一推测的正确性。那么，显式解的渐近阶即为所求。
2. 迭代法 这个方法的基本步骤是通过反复迭代，将递归方程的右端变换成一个级数，然后求级数的和，再估计和的渐近阶；或者，不求级数的和而直接估计级数的渐近阶，从而达到对递归方程解的渐近阶的估计。
3. 套用公式法 这个方法针对形如：$T(n)=aT(n/b)+f(n)$ **的递归方程，**给出三种情况下方程解的渐近阶的三个相应估计公式供套用。
4. 差分方程法 有些递归方程可以看成一个差分方程，因而可以用解差分方程(初值问题)的方法来解递归方程。然后对得到的解作渐近阶的估计。
5. 母函数法 这是一个有广泛适用性的方法。它不仅可以用来求解线性常系数高阶齐次和非齐次的递归方程，而且可以用来求解线性变系数高阶齐次和非齐次的递归方程，甚至可以用来求解非线性递归方程。方法的基本思想是设定递归方程解的母函数，努力建立一个关于母函数的可解方程，将其解出，然后返回递归方程的解。

本章将逐一地介绍上述五种方法，并分别举例加以说明。

本来，递归方程都带有初始条件，为了简明起见，我们在下面的讨论中略去这些初始条件。

递归方程组解的渐进阶的求法——代入法

用这个办法既可估计上界也可估计下界。如前面所指出，方法的关键步骤在于预先对解答作出推测，然后用数学归纳法证明推测的正确性。

例如，我们要估计 T(n)的上界，T(n)满足递归方程：

$$T(n) = 2T(\lfloor n/2 \rfloor) + n \qquad (6.1)$$

其中$\lfloor\ \rfloor$是地板(floors)函数的记号，$\lfloor n \rfloor$表示不大于 n 的最大整数。

我们推测 T(n)=O(n*log* n)，即推测存在正的常数 C 和自然数 n_0，使得当 $n \geqslant n_0$ 时有：

$T(n) \leqslant Cn\log\ n$ (6.2)

事实上，取 $n_0=2^2=4$，并取

$$C = \max_{n_0<n<2n_0} (T(n)/(n\log n))+1$$

那么，当 $n_0 \leqslant n \leqslant 2n_0$ 时，(6.2)成立。今归纳假设当 $2^{k-1}n_0 \leqslant n \leqslant 2^k n_0$ ，$k \geqslant 1$ 时，(1.1.16)成立。那么，当 $2^k n_0 \leqslant n \leqslant 2^{k+1} n_0$ 时，我们有：

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2C\lfloor n/2 \rfloor \cdot \log(\lfloor n/2 \rfloor) + n \\ &< 2C \cdot \frac{n}{2} \cdot \log \frac{n}{2} + n \\ &= Cn\log n - Cn + n \\ &= Cn\log n - (c-1)n \\ &\leq Cn\log n \end{aligned}$$

即(6.2)仍然成立，于是对所有 $n \geqslant n_0$，(6.2)成立。可见我们的推测是正确的。因而得出结论：递归方程(6.1)的解的渐近阶为 $O(n\log n)$。

这个方法的局限性在于它只适合容易推测出答案的递归方程或善于进行推测的高手。推测递归方程的正确解，没有一般的方法，得靠经验的积累和洞察力。我们在这里提三点建议：

(1) 如果一个递归方程类似于你从前见过的已知其解的方程，那么推测它有类似的解是合理的。作为例子，考虑递归方程：

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n \qquad (6.3)$$

右边项的变元中加了一个数 17，使得方程看起来难于推测。但是它在形式上与(6.1)很类似。实际上，当 n 充分大时

$T(\lfloor n/2 \rfloor + 17) + n$ 与 $T(\lfloor n/2 \rfloor)$

相差无几。因此可以推测(6.3)与(6.1)有类似的上界 $T(n)=O(n\log n)$。进一步，数学归纳将证明此推测是正确的。

(2)从较宽松的界开始推测，逐步逼近精确界。比如对于递归方程(6.1)，要估计其解的渐近下界。由于明显地有 $T(n)\geqslant n$，我们可以从推测 $T(n)=\Omega(n)$开始，发现太松后，把推测的阶往上提，就可以得到 $T(n)=\Omega(n\log n)$的精确估计。

(3)作变元的替换有时会使一个末知其解的递归方程变成类似于你曾见过的已知其解的方程，从而使得只要将变换后的方程的正确解的变元作逆变换，便可得到所需要的解。例如考虑递归方程：

$$T(n)=2T(\lfloor\sqrt{n}\rfloor)+\log n \qquad (6.4)$$

看起来很复杂，因为右端变元中带根号。但是，如果作变元替换 $m=\log n$，即令 $n=2^m$，将其代入(6.4)，则(6.4)变成：

$$T(2^m)=2T(\lfloor 2^{m/2}\rfloor)+m \qquad (6.5)$$

把 m限制在正偶数集上，则(6.5)又可改写为：

$$T(2^m)=2T(2^{m/2})+m$$

若令 $S(m)=T(2^m)$，则 $S(m)$满足的递归方程：

$$S(m)=2S(m/2)+m \ ,$$

与(6.1)类似，因而有：

$$S(m)=O(m\log m),$$

进而得到 $T(n)=T(2^m)=S(m)=O(m\log m)=O(\log n\log\log n)$ (6.6)

上面的论证只能表明：当(充分大的)n是 2 的正偶次幂或换句话说是 4 的正整数次幂时(6.6)才成立。进一步的分析表明(6.6)对所有充分大的正整数 n都成立，从而，递归方程(6.4)解的渐近阶得到估计。

在使用代入法时，有三点要提醒：

(1)记号 O不能滥用。比如,在估计(6.1)解的上界时,有人可能会推测 $T(n)=O(n)$，即对于充分大的 n，有 $T(n)\leqslant Cn$ ，其中 C是确定的正的常数。他进一步运用数学归纳法，推出：

$$T(n)=2T(\lfloor n/2\rfloor)+n\leq 2c\lfloor n/2\rfloor+n<2C\cdot n/2+n=C\cdot n+n=(C+1)\cdot n=O(n)$$

从而认为推测 $T(n)=O(n)$是正确的。实际上，这个推测是错误的，原因是他滥用了记号 O ，错误地把$(C+1)n$与 Cn等同起来。

(2)当对递归方程解的渐近阶的推测无可非议，但用数学归纳法去论证又通不过时，不妨在原有推测的基础上减去一个低阶项再试试。作为一个例子，考虑递归方程

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/2 \rfloor) + 1 \qquad (6.7)$$

其中$\lfloor\ \rfloor$是天花板(floors)函数的记号。我们推测解的渐近上界为 $O(n)$。我们要设法证明对于适当选择的正常数 C和自然数 n_0，当 $n \geqslant n_0$时有 $T(n) \leqslant Cn$。把我们的推测代入递归方程，得到：

$$T(n) = C\lfloor n/2 \rfloor + C\lfloor n/2 \rfloor + 1 = C(\lfloor n/2 \rfloor + \lfloor n/2 \rfloor) + 1 = Cn + 1 \qquad (6.8)$$

我们不能由此推断 $T(n) \leqslant Cn$，归纳法碰到障碍。原因在于(6.8)的右端比 Cn 多出一个低阶常量。为了抵消这一低阶量，我们可在原推测中减去一个待定的低阶量 b，即修改原来的推测为 $T(n) \leqslant Cn-b$ 。现在将它代人(6.7)，得到：

$$T(n) \leq (C\lfloor n/2 \rfloor - b) + (C\lfloor n/2 \rfloor - b) + 1 = C(\lfloor n/2 \rfloor + \lfloor n/2 \rfloor) - 2b + 1 = Cn - 2b + 1 \leq Cn - b$$

只要 $b \geqslant 1$，新的推测在归纳法中将得到通过。

(3)因为我们要估计的是递归方程解的渐近阶，所以不必要求所作的推测对递归方程的初始条件（如 $T(0)$、$T(1)$）成立，而只要对 $T(n)$成立，其中 n 充分大。比如，我们推测(6.1)的解 $T(n) \leqslant Cn\log n$，而且已被证明是正确的，但是当 $n=1$ 时，这个推测却不成立，因为$(Cn\log n)|_{n=1}=0$ 而 $T(1)>0$。

另外一种方式

例：Hanoi 塔问题递归算法的时间复杂性，由以下递归方程给出：

$$\begin{cases} T(n) = 2T(n-1) + 1 & n \geq 2 \\ T(1) = 1 \end{cases}$$

递推求解如下：

$$
\begin{aligned}
T(n) &= 2T(n-1)+1 \\
&= 2(2T(n-2)+1)+1 \\
&= 2^2T(n-2)+2+1 \\
&= 2^3T(n-3)++2^2+2+1 \\
&\cdots\cdots \\
&= 2^{n-1}T(1)+2^{n-2}+\ldots+2^2+2+1 \\
&= 2^{n-1}+2^{n-2}+\ldots+2^2+2+1 \\
&= 2^n-1
\end{aligned}
$$

所以，Hanoi 塔问题递归算法的时间复杂性为：$T(n)=O(2^n)$

递归方程组解的渐进阶的求法——迭代法

用这个方法估计递归方程解的渐近阶不要求推测解的渐近表达式，但要求较多的代数运算。方法的思想是迭代地展开递归方程的右端，使之成为一个非递归的和式，然后通过对和式的估计来达到对方程左端即方程的解的估计。

作为一个例子，考虑递归方程：

$$T(n) = 3T(\lfloor n/4 \rfloor) + n \qquad (6.9)$$

接连迭代二次可将右端项展开为：

$$\begin{aligned} T(n) &= 3T(\lfloor n/4 \rfloor) + n \\ &= n + 3(\lfloor n/4 \rfloor + 3(T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) \\ &= n + 3(\lfloor n/4 \rfloor + 3(\lfloor \lfloor n/4 \rfloor /4 \rfloor + 3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor))) \end{aligned} \qquad (6.10)$$

由于对地板函数有恒等式：

$$\lfloor \lfloor n/a \rfloor /b \rfloor = \lfloor n/ab \rfloor$$

(6.10)式可化简为：

$$T(n) = n + 3(\lfloor n/4 \rfloor + 3(\lfloor n/4^2 \rfloor + 3T(\lfloor n/4^3 \rfloor)))$$

这仍然是一个递归方程，右端项还应该继续展开。容易看出，迭代 i 次后，将有

$$T(n) = n + 3(\lfloor n/4 \rfloor + 3(\lfloor n/4^2 \rfloor + 3T(\lfloor n/4^3 \rfloor + \cdots + 3(\lfloor n/4^i \rfloor + 3T(\lfloor n/4^{i+1} \rfloor)\cdots))) \quad (6.11)$$

而且当

$$\lfloor n/4^{i+1} \rfloor = 0 \qquad (6.12)$$

时，(6.11)不再是递归方程。这时：

$$T(n) = n + 3(\lfloor n/4 \rfloor + 3(\lfloor n/4^2 \rfloor + 3(\lfloor n/4^3 \rfloor + \cdots + 3(\lfloor n/4^i \rfloor + 3T(0))\cdots))) \quad (6.13)$$

又因为$[a] \leqslant a$，由(6.13)可得：

$$T(n) \le n+3(n/4+3(n/4^2+3(n/4^3+\cdots+3(n/4^i+3T(0))\cdots)))$$
$$=n+\frac{3}{4}n+\frac{3^2}{4^2}n+\frac{3^3}{4^3}n+\cdots+\frac{3^i}{4^i}n+3^{i+1}\cdot T(0)$$
$$<4n+3^{i+1}\cdot T(0) \qquad (6.14)$$

而由(6.12)，知 $i\leqslant\log_4 n$ ，从而

$$3^{i+1}\le 3^{\log_4 n+1}=3^{\log_3 n\cdot\log_4 3+1}=3n^{\log_4 3},$$

代人(6.14)得：

$$T(n)<4n+3n^{\log_4 3}\cdot T(0)$$

即方程(6.9)的解　　$T(n)=O(n)$。

递归树

从这个例子可见迭代法导致繁杂的代数运算。但认真观察一下，要点在于确定达到初始条件的迭代次数和抓住每次迭代产生出来的"自由项"(与 T无关的项)遵循的规律。顺便指出，迭代法的前几步迭代的结果常常能启发我们给出递归方程解的渐近阶的正确推测。这时若换用代入法，将可免去上述繁杂的代数运算。

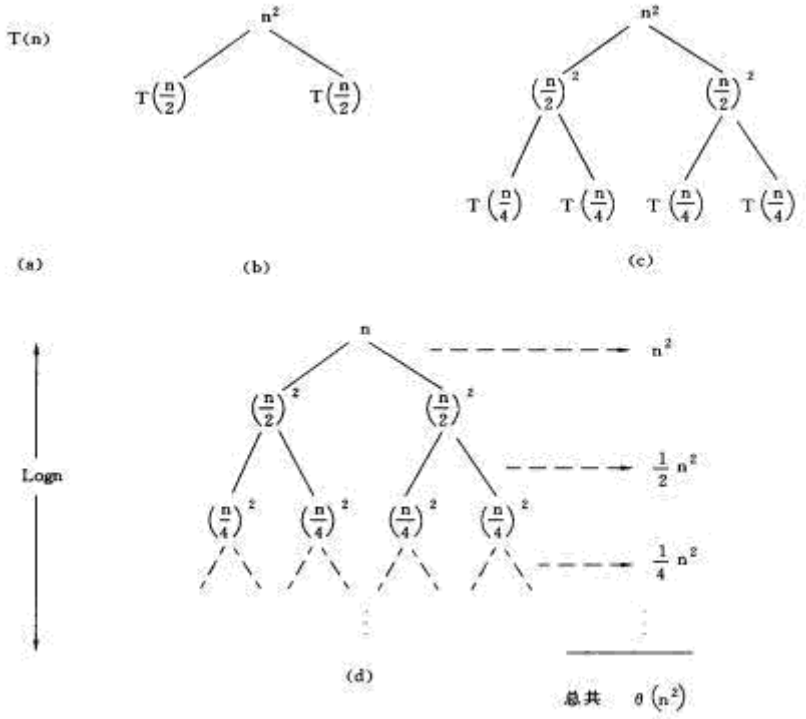


图 6-1 与方程(6.15)相应的递归树

为了使迭代法的步骤直观简明、图表化，我们引入*递归树*。靠着递归树，人们可以很快地得到递归方程解的渐近阶。它对描述分治算法的递归方程特别有效。我们以递归方程

$$T(n)=2T(n/2)+n^2 \quad (6.15)$$

为例加以说明。图 6-1 展示出(6.15)在迭代过程中递归树的演变。为了方便，我们假设 n 恰好是 2 的幂。在这里，递归树是一棵二叉树，因为(6.15)右端的递归项 $2T(n/2)$ 可看成 $T(n/2)+T(n/2)$。图 6-1(a)表示 $T(n)$ 集中在递归树的根处，(b)表示 $T(n)$ 已按(6.15)展开。也就是将组成它的自由项 n^2 留在原处，而将 2 个递归项 $T(n/2)$ 分别摊给它的 2 个儿子结点。(c)表示迭代被执行一次。图 6-1(d)展示出迭代的最终结果。

图 6-1 中的每一棵递归树的所有结点的值之和都等于 $T(n)$。特别，已不含递归项的递归树(d)中所有结点的值之和亦然。我们的目的是估计这个和 $T(n)$。我们看到有一个表格化的办法：先按横向求出每层结点的值之和，并记录在各相应层右端顶格处，然后从根到叶逐层地将顶格处的结果加起来便是我们要求的结果。照此，我们得到(6.15)解的渐近阶为 $\theta(n^2)$。

再举一个例子。递归方程：

$$T(n)= T(n/3)+ T(2n/3)+n \quad (6.16)$$

的迭代过程相应的递归树如图 6-2 所示。其中，为了简明，再一次略去地板函数和天花板函数。

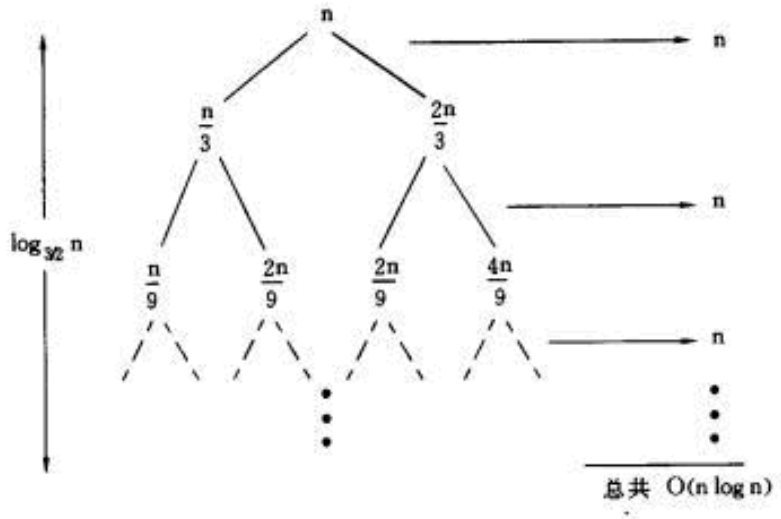


图 6-2 迭代法解(6.16)的递归树

当我们累计递归树各层的值时，得到每一层的和都等于 n，从根到叶的最长路径是

$$n \to \frac{2}{3}n \to (\frac{2}{3})^2 n \to \cdots \to 1$$

设最长路径的长度为 k，则应该有

$$(\frac{2}{3})^k n = 1$$

，

得

$$k = \log_{3/2} n$$

，

于是

$$T(n) \le \sum_{i=0}^{k} n = (k+1)n = n(\log_{3/2} n + 1)$$

即 $T(n)=O(n\log n)$ 。

以上两个例子表明，借助于递归树，迭代法变得十分简单易行。

递归方程组解的渐进阶的求法——套用公式法

这个方法为估计形如：

$T(n)=aT(n/b)+f(n)$ (6.17)

的递归方程解的渐近阶提供三个可套用的公式。(6.17)中的 $a\geqslant 1$ 和 $b\geqslant 1$ 是常数，$f(n)$ 是一个确定的正函数。

(6.17)是一类分治法的时间复杂性所满足的递归关系，即一个规模为 n 的问题被分成规模均为 n/b 的 a 个子问题，递归地求解这 a 个子问题，然后通过对这 a 个子间题的解的综合，得到原问题的解。如果用 $T(n)$ 表示规模为 n 的原问题的复杂性，用 $f(n)$ 表示把原问题分成 a 个子问题和将 a 个子问题的解综合为原问题的解所需要的时间，我们便有方程(6.17)。

这个方法依据的是如下的定理：设 $a\geqslant 1$ 和 $b\geqslant 1$ 是常数 $f(n)$ 是定义在非负整数上的一个确定的非负函数。又设 $T(n)$ 也是定义在非负整数上的一个非负函数，且满足递归方程(6.17)。方程(6.17)中的 n/b 可以是 $[n/b]$，也可以是 n/b。那么，在 $f(n)$ 的三类情况下，我们有 $T(n)$ 的渐近估计式：

1. 若对于某常数 $\varepsilon>0$，有

   $$f(n)=O(n^{\log_b a-\varepsilon}),$$

   则

   $$T(n)=\theta(n^{\log_b a});$$

2. 若

   $$f(n)=\theta(n^{\log_b a}),$$

   则

   $$T(n)=\theta(n^{\log_b a}\cdot\log n);$$

3. 若对其常数 $\varepsilon>0$，有

   $$f(n)=\Omega(n^{\log_b a+\varepsilon})$$

   且对于某常数 $c>1$ 和所有充分大的正整数 n 有 $af(n/b)\leq cf(n)$，则 $T(n)=\theta(f(n))$。

这里省略定理的证明。

在应用这个定理到一些实例之前，让我们先指出定理的直观含义，以帮助读者理解这个定理。读者可能已经注意到，这里涉及的三类情况，都是拿 $f(n)$ 与 $n^{\log_a b}$ 作比较。定理直观地告诉我们，递归方程解的渐近阶由这两个函数中的较大者决

定。在第一类情况下，函数$n^{\log_a b}$较大，则 $T(n)=\theta(n^{\log_a b})$；在第三类情况下，函数 $f(n)$较大，则 $T(n)=\theta(f(n))$；在第二类情况下，两个函数一样大，则 $T(n)=\theta(n^{\log_a b}\cdot\log n)$，即以 n 的对数作为因子乘上 $f(n)$与 $T(n)$的同阶。

此外，定理中的一些细节不能忽视。在第一类情况下 $f(n)$不仅必须比$n^{\log_a b}$小，而且必须是多项式地比　　小，即 $f(n)$必须渐近地小于$n^{\log_a b}$与$n^{-\varepsilon}$的积，ε是一个正的常数；在第三类情况下 $f(n)$不仅必须比$n^{\log_a b}$大，而且必须是多项式地比$n^{\log_a b}$大，还要满足附加的“正规性”条件：$af(n/b)\leqslant cf(n)$。这个附加的“正规性”条件的直观含义是 a 个子问题的再分解和再综合所需要的时间最多与原问题的分解和综合所需要的时间同阶。我们在一般情况下将碰到的以多项式为界的函数基本上都满足这个正规性条件。

还有一点很重要，即要认识到上述三类情况并没有覆盖所有可能的 $f(n)$。在第一类情况和第二类情况之间有一个间隙：$f(n)$小于但不是多项式地小于$n^{\log_a b}$；类似地，在第二类情况和第三类情况之间也有一个间隙：$f(n)$大于但不是多项式地大于$n^{\log_a b}$。如果函数 $f(n)$落在这两个间隙之一中，或者虽有$f(n)=\Omega(n^{\log_b a+\varepsilon})$，但正规性条件不满足，那么，本定理无能为力。

下面是几个应用例子。

例 1 考虑

$T(n)=9T(n/3)+n$。

对照(6.17)，我们有 $a=9$，$b=3$，　$f(n)=n$，　$n^{\log_b a}=n^{\log_3 9}=n^2$，取$\varepsilon\in(0,1]$，便有$f(n)=O(n^{\log_b a-\varepsilon})$，可套用第一类情况的公式，得 $T(n)=\theta(n^2)$。

例 2 考虑

T(n)=T(2n/3)+1

对照(6.17)，我们有 a=1，b=3/2，　f(n)=1，$n^{\log_b a}=n^{\log_{3/2}1}=n^0=1=f(n)$，可套用第二类情况的公式，得 T(n)=θ(1ogn)。

例 3 考虑

T(n)=3T(n/4)+nlogn

对照(6.17)，我们有 a=3，b=4， f(n)=nlog n， $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$，只要取 $\varepsilon \approx 0.2$，便有 $f(n) = \Omega(n^{\log_b a+\varepsilon})$。进一步，检查正规性条件：

$$af(n/b) = 3f(n/4) = 3(n/4)\log(n/4) = \frac{3}{4}n(\log n - \log 4) \le \frac{3}{4}n\log n$$

只要取 c=3/4，便有 af(n/b)≤cf(n)，即正规性条件也满足。可套用第三类情况的公式，得 T(n)= θ (f(n))= θ (nlogn)。

最后举一个本方法对之无能为力的例子。

考虑

$T(n)=2T(n/2)+n\log n$

对照(6.17)，我们有 a=2，b=2， $f(n)=n\log n$， $n^{\log_b a} = n$，虽然 $f(n)$渐近地大于 $n^{\log_a b}$，但 $f(n)$并不是多项式地大于 $n^{\log_a b}$，因为对于任意的正常数 ε，

$$f(n)/n^{\log_b a+\varepsilon} = n\log n / n^{1+\varepsilon} = n^{-\varepsilon}\log n \to 0,$$

即 $f(n)$在第二类情况与第三类情况的间隙里，本方法对它无能为力。

递归方程组解的渐进阶的求法——差分方程法

这里只考虑形如：

$$T(n)=c_1T(n-1)+c_2T(n-2)+\cdots+c_kT(n-k)+f(n),\quad n\geqslant k \quad (6.18)$$

的递归方程。其中 c_i (i=1，2，…，k)为实常数，且 $c_k\neq0$。它可改写为一个线性常系数 k 阶非齐次的差分方程：

$$T(n)-c_1T(n-1)-c_2T(n-2)-\cdots-c_kT(n-k)=f(n),\quad n\geqslant k \quad (6.19)$$

(6.19)与线性常系数 k 阶非齐次常微分方程的结构十分相似，因而解法类同。限于篇幅，这里直接给出(6.19)的解法，略去其正确性的证明。

第一步，求(6.19)所对应的齐次方程：

$$T(n)-c_1T(n-1)-c_2T(n-2)-\cdots-c_kT(n-k)=0 \quad (6.20)$$

的基本解系：写出(6.20)的特征方程：

$$C(t)=t^k-c_1t^{k-1}-c_2t^{k-2}-\cdots-c_k=0 \quad (6.21)$$

若 $t=r$ 是(6.21)的 m 重实根，则得(6.20)的 m 个基础解 r^n，nr^n，n^2r^n，…，$n^{m-1}r^n$；若 $\rho e^{i\theta}$ 和 $\rho e^{-i\theta}$ 是(6.21)的一对 l 重的共扼复根，则得(6.20)的 2l 个基础解 $\rho^n\cos n\theta$，$\rho^n\sin n\theta$，$n\rho^n\cos n\theta$，$n\rho^n\sin n\theta$，…，$n^{l-1}\rho^n\cos n\theta$，$n^{l-1}\rho^n\cos n\theta$。如此，求出(6.21)的所有的根，就可以得到(6.20)的 k 个的基础解。而且，这 k 个基础解构成了(6.20)的基础解系。即(6.20)的任意一个解都可以表示成这 k 个基础解的线性组合。

第二步，求(6.19)的一个特解。理论上，(6.19)的特解可以用 Lagrange 常数变易法得到。但其中要用到(6.20)的通解的显式表达，即(6.20)的基础解系的线性组合，十分麻烦。因此在实际中，常常采用试探法，也就是根据 $f(n)$的特点推测特解的形式，留下若干可调的常数，将推测解代人(6.19)后确定。由于(6.19)的特殊性，可以利用迭加原理，将 $f(n)$线性分解为若干个单项之和并求出各单项相应的特解，然后迭加便得到 $f(n)$相应的特解。这使得试探法更为有效。为了方便，这里对三种特殊形式的 $f(n)$，给出(6.19)的相应特解并列在表 6-1 中，可供直接套用。其中 p_i，i=1,2,…,s 是待定常数。

表 6-1 方程(6.19)的常用特解形式

$f(n)$的形式	**条　件**	**方程(6.19)的特解的形式**

a^n	$C(a)\neq 0$	$p_0 a^n$
	a 是 $C(t)$的 m 重根	$p_0 \cdot n^m \cdot a^n$
n^s	$C(1)\neq 0$	$p_0 + p_1 \cdot n + p_2 \cdot n^2 + \cdots + p_s \cdot n^s$
	1 是 $C(t)$的 m 重根	$n^m \cdot (p_0 + p_1 \cdot n + p_2 \cdot n^2 + \cdots + p_s \cdot n^s)$
$n^s a^n$	$C(a)\neq 0$	$(p_0 + p_1 \cdot n + p_2 \cdot n^2 + \cdots + p_s \cdot n^s) \cdot a^n$
	a 是 $C(t)$的 m 重根	$n^m \cdot (p_0 + p_1 \cdot n + p_2 \cdot n^2 + \cdots + p_s \cdot n^s) \cdot a^n$

第三步，写出(6.19)即(6.18)的通解

$$T(n) = \sum_{i=0}^{k-1} a_i T_i(n) + g(n) \qquad (6.22)$$

其中$\{T_i(n)$， i=0, 1, 2, …, $n\}$是(6.20)的基础解系，$g(n)$是(6.19)的一个特解。然后由(6.18)的初始条件

$T(i)=T_i$ ， i=1, 2, …, k-1

来确定(6.22)中的待定的组合常数$\{a_i\}$，即依靠线性方程组

$$\sum_{i=0}^{k-1} a_i T_i(j) + g(j) = T_j, \qquad j = 0,1,2,\cdots,k-1$$

或

$$\sum_{i=0}^{k-1} a_i T_i(j) = \beta_j, \qquad j = 0,1,2,\cdots,k-1$$

解出$\{a_i\}$，并代回(6.22)。其中 $\beta_j=T_j-g(j)$， j=0, 1, 2, …, k-1。

第四步，估计(6.22)的渐近阶，即为所要求。

下面用两个例子加以说明。

例 1 考虑递归方程

$$F(n)=\begin{cases}2 & n=0\\ 3 & n=1\\ 8+F(n-1)+F(n-2) & n>1\end{cases}$$

它的相应特征方程为：

$C(t)=t^2-t-1=0$

解之得两个单根$r_0=(1-\sqrt{5})/2$和$r_0=(1+\sqrt{5})/2$。相应的(6.20)的基础解系为$\{r_0^n,\ r_1^n\}$。相应的(6.19)的一个特解为 $F^*(n)=-8$，因而相应的(6.19)的通解为：

$F(n)=a_0r_0^n+a_1r_1^n-8$

令其满足初始条件，得二阶线性方程组：

$$\begin{cases}a_0+a_1-8=2\\ a_0r_0+a_1r_1-8=3\end{cases}$$

或

$$\begin{cases}a_0+a_1=10\\ (a_0+a_1)/2+(-a_0+a_1)\cdot\sqrt{5}/2=11\end{cases}$$

或

$$\begin{cases}a_0+a_1=10\\ -a_0+a_1=12\sqrt{5}/5\end{cases}$$

解之得$a_0=5-6\sqrt{5}/5$，$a_1=5+6\sqrt{5}/5$，从而

$$F(n)=(5-6\sqrt{5}/5)(\frac{1-\sqrt{5}}{2})^n+(5+6\sqrt{5}/5)(\frac{1+\sqrt{5}}{2})^n-8$$

于是

$F(n)=\theta((\frac{1+\sqrt{5}}{2})^n)$。

例 2 考虑递归方程

$T(n)=4T(n-1)-4T(n-2)+2^n n$ (6.23)

和初始条件 $T(0)=0$， $T(1)=4/3$。

它对应的特征方程(6.21)为

$C(t)=t^2-4t+4=0$

有一个两重根 $r=2$。故相应的(6.20)的基础解系为$\{2^n, 2^n n\}$。由于 $f(n)=2^n n$，利用表 6-1，相应的(6.19)的一个特解为

$T^*(n)=n^2(p_0+p_1 n)2^n$，

代人(6.23)，定出 $p_0=1/2$， $p_1=1/6$。因此相应的(6.19)的通解为：

$T(n)=a_0 2^n+a_1 n 2^n+n^2(1/2+n/6)2^n$，

令其满足初始条件得 $a_0=a_1=0$，从而

$T(n)=n^2(1/2+n/6)2^n$

于是 $T(n)=\theta(n^3 2^n)$。

递归方程组解的渐进阶的求法——母函数法

关于 $T(n)$的递归方程的解的母函数通常设为：

$$A(x)=\sum_{n=0}^{\infty}T(n)x^n \qquad (6.24)$$

当(6.24)右端由于 $T(n)$增长太快而仅在 $x=0$ 处收敛时可另设

$$A(x)=\sum_{n=0}^{\infty}T(n)x^n/n! \qquad (6.25)$$

如果我们可以利用递归方程建立 $A(x)$的一个定解方程并将其解出，那么，把 $A(x)$展开成幂级数，则 x^n或 $x^n/n!$项的系数便是所求的递归方程的解。其渐近阶可接着进行估计。

下面举两个例子加以说明。

例 1 考虑线性变系数二阶齐次递归方程

$(n-1)T(n)=(n-2)T(n-1)+2T(n-2)$ ， $n\geq 2$ (6.26)

和初始条件 $T(0)=0$， $T(1)=1$。根据初始条件及(6.26)，可计算 $T(2)=0$，$T(3)=T(1)=1$。

设$\{T(n)\}$的母函数为：

$$A(x)=\sum_{n=0}^{\infty}T(n)x^n$$

由于 $T(0)=T(2)=0$， $T(1)=1$，有 ：

$$\begin{aligned}A(x)&=x+T(3)x^3+T(4)x^4+\cdots+T(n)x^n+\cdots\\&=x(1+T(3)x^2+T(4)x^3+\cdots+T(n)x^{n-1}+\cdots)\end{aligned}$$

令 $B(x)=A(x)/x$，即：

$$B(x)=1+T(3)x^2+T(4)x^3+\cdots+T(n)x^{n-1}+\cdots$$

那么：

$$B'(x)=2T(3)x+3T(4)x^2+\cdots+(n-1)T(n)x^{n-2}+\cdots$$
$$xB'(x)=2T(3)x^2+3T(4)x^3+\cdots+(n-1)T(n)x^{n-1}+\cdots$$
$$B'(x)-xB'(x)=2T(3)x+(3T(4)-2T(3))x^2+(4T(5)-3T(4))x^3+\cdots+((n-1)T(n)-(n-2)T(n-1))x^{n-2}+\cdots$$

利用(6.26)并代入 T (3)= 1，得

$$\begin{aligned}B'(x)(1-x)&=2T(1)x+2T(2)x^2+2T(3)x^3+\cdots+2T(n-2)x^{n-2}+\cdots\\&=2x(1+T(3)x^2+T(4)x^3+\cdots+T(n)x^{n-1}+\cdots)\\&=2xB(x)\end{aligned}$$

即

$$\frac{B'(x)}{B(x)}=\frac{2x}{1-x}=-2+\frac{2}{1-x}$$

两边同时沿[0, x]积分，并注意到 $B(0)$=1，有：

$$\ln B(x)=-2x-2\ln(1-x)$$
$$B(x)=e^{-2x}/(1-x)^2$$

把 $B(x)$展开成幂级数，得

$$\begin{aligned}B(x)&=\sum_{n=0}^{\infty}\frac{(-2x)^n}{n!}\sum_{n=0}^{\infty}\binom{n+1}{1}x^n\\&=\sum_{i=0}^{\infty}\frac{(-1)^i\cdot 2^i}{i!}x^i\cdot\sum_{j=0}^{\infty}(j+1)\cdot x^j\\&=\sum_{i=0}^{\infty}\sum_{j=0}^{\infty}\frac{(-1)^i\cdot 2^i\cdot(j+1)}{i!}x^{(i+j)}\\&=\sum_{n=0}^{\infty}(\sum_{i=0}^{n}\frac{(-1)^i\cdot 2^i\cdot(n-i+1)}{i!})\cdot x^n\qquad 0\le x<1\end{aligned}$$

从而

$$\begin{aligned}A(x)&=\sum_{n=0}^{\infty}(\sum_{i=0}^{n}\frac{(-1)^i\cdot 2^i\cdot(n-i+1)}{i!})\cdot x^{n+1}\\&=\sum_{n=1}^{\infty}(\sum_{i=0}^{n-1}\frac{(-1)^i\cdot 2^i\cdot(n-i)}{i!})\cdot x^n\qquad 0\le x<1\end{aligned}$$

最后得

$$T(n)=\sum_{i=1}^{n-1}\frac{(-1)^i\cdot 2^i\cdot(n-i)}{i!}\qquad n\geq 1$$

例 2 考虑线性变系数一阶非齐次递归方程

$D(n)=nD(n-1)+(-1)^n \quad n\geqslant 1$ (6.27)

及初始条件 $D(0)=1$

很明显 $D(n)$随 n 的增大而急剧增长。如果仍采用(6.24)形式的函数，则(6.24)的右端可能仅在 $x=0$ 处收敛，所以这里的母函数设为：

$$A(x)=\sum_{n=0}^{\infty}D(n)x^n/n!$$

用 $x^n/n!$乘以(6.27)的两端，然后从 1 到∞求和得：

$$\sum_{n=1}^{\infty}D(n)x^n/n!=\sum_{n=1}^{\infty}D(n-1)x^n/(n-1)!+\sum_{n=1}^{\infty}(-1)^n x^n/n!$$

化简并用母函数表达，有：

$A(x)-1=xA(x)+e^{-x}-1$

或

$(1-x)A(x)=e^{-x}$

从而

$A(x)=e^{-x}/(1-x)$

展成幂级数，则：

$$
\begin{aligned}
A(x) &= \sum_{i=0}^{\infty} (-1)^i x^i / i! \sum_{j=0}^{\infty} x^j \\
&= \sum_{i=0}^{\infty} \sum_{j=0}^{\infty} ((-1)^i / i!) x^{i+j} \\
&= \sum_{n=0}^{\infty} (\sum_{i=0}^{n} (-1)^i / i!) x^n \\
&= \sum_{n=0}^{\infty} n! (\sum_{i=0}^{n} (-1)^i / i!) x^n / n!
\end{aligned}
$$

故

$$
D(n) = n! \sum_{i=0}^{n} (-1)^i / i!
$$